

Trilha de Docker

*Módulo 10: Otimizando e
Debugando Contêineres*

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.
2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.
3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.
4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.
5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.
6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.
7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.
8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Otimizando e Debugando Contêineres

Conceito

Em projetos reais, é comum precisar **identificar erros**, entender por que um contêiner não inicia e **reduzir o tamanho das imagens** para acelerar builds e deploys.

Para isso, o Docker oferece recursos como:

- **Logs** do contêiner (para investigar falhas)
 - **Shell interativo** (para inspecionar arquivos e processos)
 - Boas práticas de Dockerfile (para **otimização de camadas e tamanho**)
-

Exemplos práticos

1) Verificar logs de um contêiner

Use logs para entender por que um serviço falhou ou parou:

```
docker logs <container_id_ou_nome>
```

Dica: se o contêiner reinicia rapidamente, os logs são a primeira fonte de diagnóstico.

2) Acessar um contêiner em modo interativo (debug)

Para inspecionar o que existe dentro do contêiner:

```
docker exec -it <container_id_ou_nome> sh
```

🔑 Se a imagem usar Debian/Ubuntu, pode ser necessário usar **bash**:

```
docker exec -it <container_id_ou_nome> bash
```

Desafios (Aplicação prática e integradora)

Desafio 1: Ambiente com PHP + Nginx + MySQL + Composer (com Docker Compose)

Objetivo

Criar um ambiente completo para uma aplicação PHP, com servidor web (Nginx), PHP-FPM, banco MySQL e Composer para dependências.

Requisitos mínimos

- Serviços separados em `docker-compose.yml`:
 - `nginx` (servidor web)
 - `php` (PHP-FPM)
 - `mysql` (banco de dados)
- Arquivo `nginx.conf` configurado para apontar para o PHP-FPM
- `Dockerfile` do serviço `php` com Composer instalado
- Persistência do banco com `volume`

Passo a passo (orientação)

1. Crie o `docker-compose.yml` com os 3 serviços (`nginx`, `php`, `mysql`)
2. Crie um `Dockerfile` para o serviço `php` instalando:
 - PHP + extensões necessárias
 - Composer
3. Configure o `nginx.conf` para:
 - servir arquivos
 - encaminhar requisições PHP para o serviço `php`
4. Suba o ambiente com:

```
docker compose up -d --build
```

5. Valide no navegador e teste conexão com o banco.

✓ Entrega esperada: aplicação PHP acessível e banco persistindo dados.

Desafio 2: Ambiente com VueJS (build) + Nginx + API NodeJS + MySQL (com Docker Compose)

Objetivo

Criar um ambiente típico de aplicações modernas:

- Frontend Vue (buildado e servido via Nginx)
- Backend Node.js (API)
- Banco MySQL
- Comunicação via rede Docker

Requisitos mínimos

- Serviços separados em `docker-compose.yml`:
 - `frontend` (Vue)
 - `nginx` (servindo o frontend buildado)

- `backend` (Node.js API)
- `mysql` (banco)
- `nginx.conf` com:
 - rota `/` servindo o frontend
 - rota `/api` fazendo proxy para o backend
- Persistência do banco com **volume**

Passo a passo (orientação)

1. Crie o `docker-compose.yml` com os serviços (`nginx`, `backend`, `mysql`, e opcionalmente `frontend`)
2. Configure o frontend para gerar build (ex.: `dist/`)
3. Configure o Nginx para servir o build do Vue
4. Configure proxy `/api` para apontar para o serviço `backend`
5. Suba o ambiente com:

```
docker compose up -d --build
```

✅ Entrega esperada: frontend funcionando + API acessível via `/api` + banco persistente.

Referências

- Documentação oficial do Docker (CLI e containers)
- Docker Hub (imagens oficiais)